Producer

“Stop thinking, and end your problems.”
- Lao Tzu



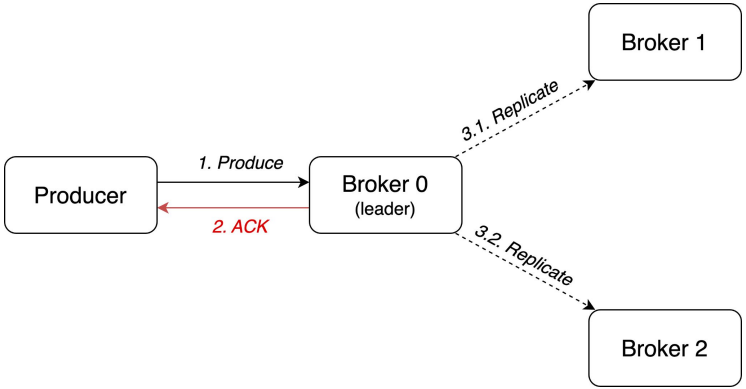
Outline

1. ACK
2. Retry
3. Batching
4. Compression
5. Sticky Partitioner
6. Max Requests In Flight
7. Idempotent

1. ACK

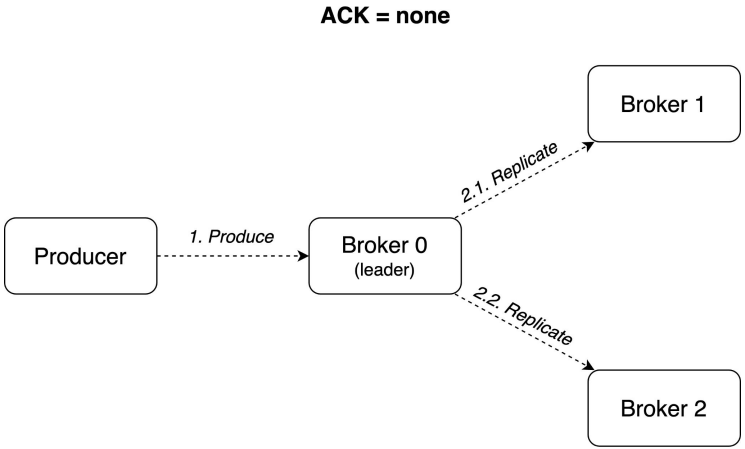
1. ACK

- Acknowledgment (ACK) is a signal sent from receiver indicating that a message has been received, and optionally, processed successfully.
- ACKS is the number of acknowledgments the producer requires the **leader to have received** before completing a request.



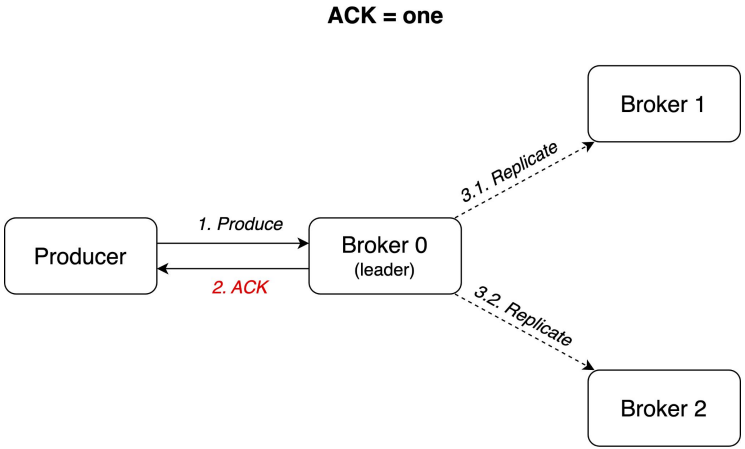
1.1. ACKS = none (0)

- Producers require **no ACK from the leader**
- Fire and forget
- Pros:
  - **Most performant**
- Cons:
  - **Durability: weakest**
  - Messages might be lost
  - No offset in the record metadata
- Use cases:
  - Some metrics from IoT devices: temperature sensor
  - Non-essential logs, …
- Records will still be replicated asynchronously



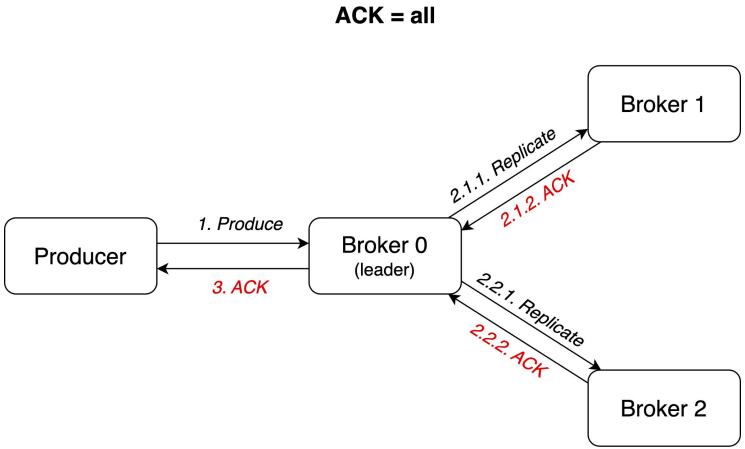
1.2. ACKS = one (1)

- Producers require **ACK from the leader only**
- Pros:
  - **High performance**
  - Offset in the record metadata
- Cons:
  - **Durability: weak**
  - Messages might be lost still
- Use cases:
  - User activity tracking
  - Tracking clicks, views, …
- Records will still be replicated asynchronously



1.3. ACKS = all (-1)

- Producers will wait until the leader has gathered ACKs **from all In-Sync Replicas** (ISRs).
- Replication here is synchronous.
- Pros:
  - **Durability: Strong**
- Cons:
  - **High latency**
- Use cases:
  - Financial activity
  - Order, payment, …

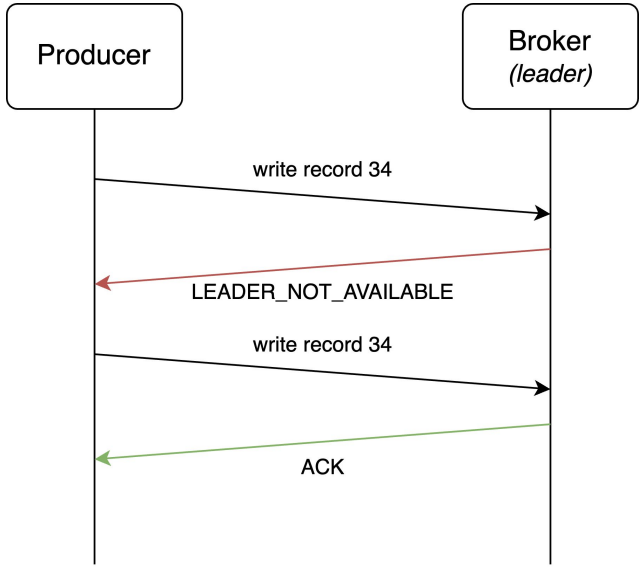


When getting an unknown error,
What should we do first?

2. Retry

2. Retry

- Producers get an unknown error while sending messages, producers should retry first.
- Types of errors:
  - **Retriable error**: LEADER_NOT_AVAILABLE, network connectivity issues, …
  - **Non-Retriable error**: INVALID_CONFIG, too large message, …
- 2 types of handling errors:
  - **Auto: automatically retry N times then give up**
  - Manual



2. Retry

- The default value of tries:
  - Kafka <= 2.0: 0
  - Kafka >= 2.1: 2147483647
- Retry.backoff.ms: 100ms
- Producers retry in 2 mins (delivery.timeout.ms=120000)
- Recommended: retries = 30
- Issues:
  - **Duplicate messages**
  - **Ordering in producer side**

What is the bottleneck of Kafka?
Disk?

Answer: Network

3. Batching

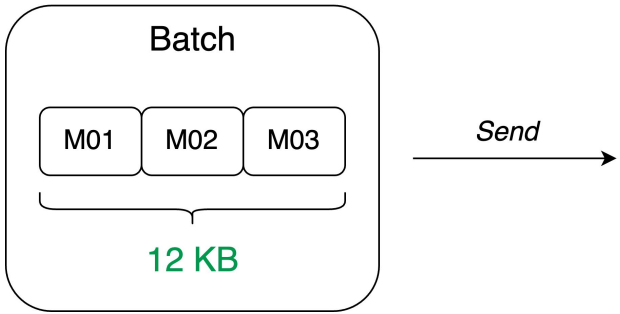
3.1. Batching

- Batching multiple records together before sending.
- The batching mechanism improves network bandwidth efficiency and throughput.
- How batching mechanism works?
- **Batching is implemented by the producer**
- A write request contains a batch
- A batch is written to a partition.
- Batching is controlled by 2 configurations:
    - **linger.ms**: A producer will wait for up to the given delay to batch.
    - **batch.size**: The maximum size of a batch.

3.2. linger.ms

- A producer will wait for up to the given delay to batch before sending.
- This adds more latency.
- The default value = 0. This means producers send records as soon as possible.
- If linger.ms = 0, batching still works.
- If **linger.ms > 0, it increases the chance of batching**.

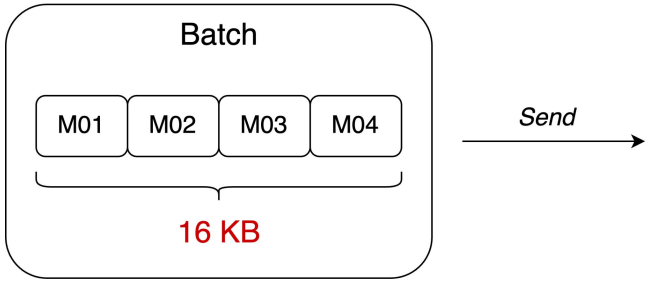
batch.size = 16 (KB)
linger.ms = 5 (ms)
delay time = 5 (ms)



3.3. batch.size

- The maximum size of a batch.
- The default value = 16 KB.
- A very large batch size may use more **memory wastefully**.
- If a message is bigger than the batch size, then the message will not be batched.

batch.size = 16 (KB)
linger.ms = 5 (ms)
delay time = 3.5 (ms)



3.4. Batching

- **Trade-off: throughput vs latency**.
- **Brokers persist a batch without unpacking the batch.**
  → improve disk I/O efficiency
- Some specific cases, brokers might modify the batch.

Batches can be big.
How to handle with something big?

- **Compress**
- Split

4. Compression

4.1. Compression

- Compression has a positive impact on performance
  - Improve network bandwidth
  - Save storage space
  - Better latency
- Batching makes the compression ratio higher
  - For JSON, the compression ratio is from 5x to 7x.
- Cons: CPU utilisation of the producer and consumer
- Note: Compression != Encryption



4.2. Compression Type

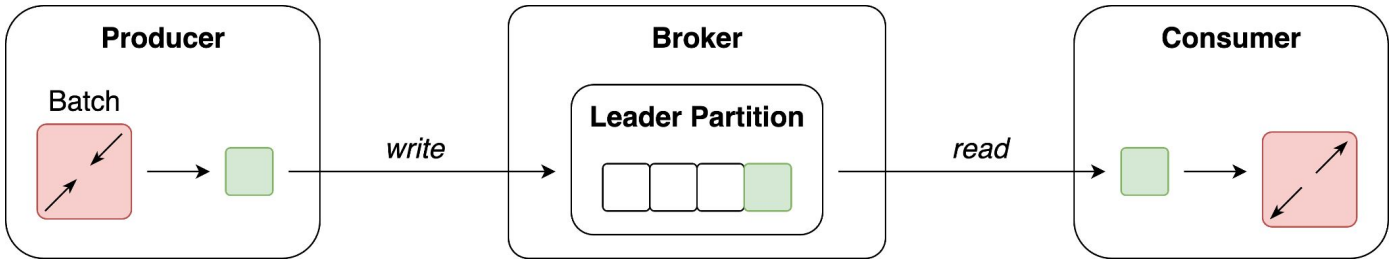
- Types:
  - **None**: (default) Compression is disabled.
  - **gzip**: Use the GNU Gzip algorithm — released in 1992, used by early UNIX systems.
  - **snappy**: Use Google's Snappy compression format — optimised for throughput at the expense of compression ratios.
  - **lz4**: Use the LZ4 algorithm — also optimised for throughput, most notably for the speed of decompression.
  - **zstd**: Use Facebook's ZStandard — a newer algorithm introduced in Kafka 2.1.0, intended to achieve an effective balance between throughput and compression ratios
- Recommend value:
  - **lz4: normal cases**
  - **zstd: very high throughput**

4.2. Compression Type

Compression Type	Compression Ratio	CPU Usage	Compression Speed	Network Bandwidth
gzip	Highest	Highest	Slowest	Slowest
snappy	Medium	Medium	Medium	Medium
lz4	Slowest	Slowest	Fastest	Highest
zstd	Medium	Medium	Medium	Medium

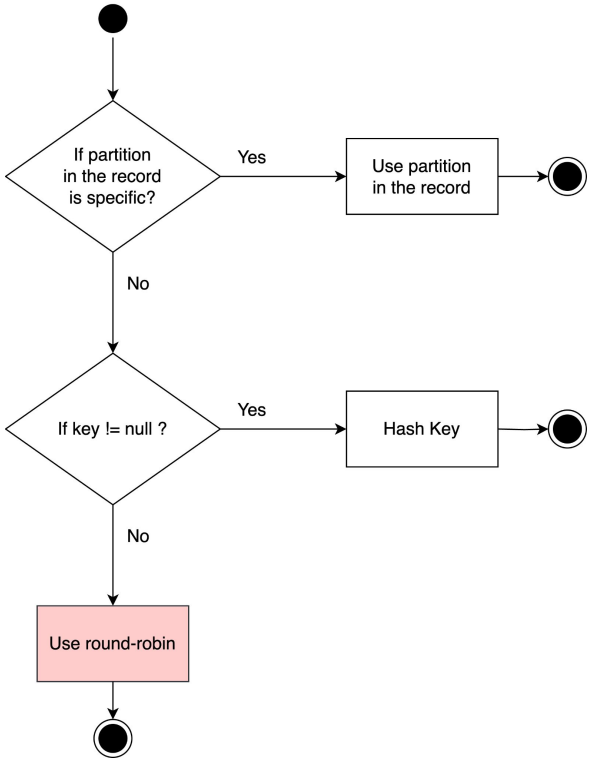
4.3. End-to-End Concern

- **Compression is implemented by producers** and **decompression is implemented by consumers**.
- By default, **brokers do not interfere with the batch** when storing the batch. No need to change the compression configuration in brokers, or consumers.
- A compression format on the producer side must be compatible with the consumer version.



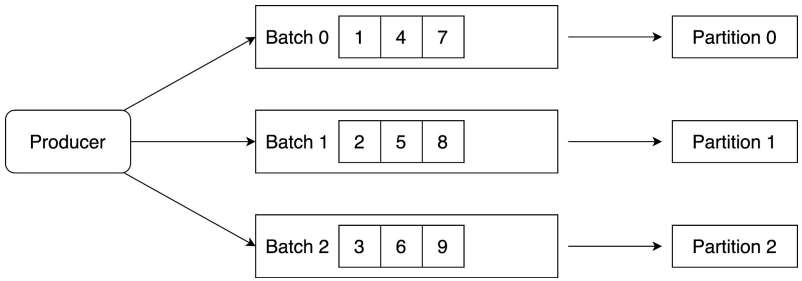
5. Sticky Partitioner

5.1. Old Default Partitioner (version < 2.4.0)



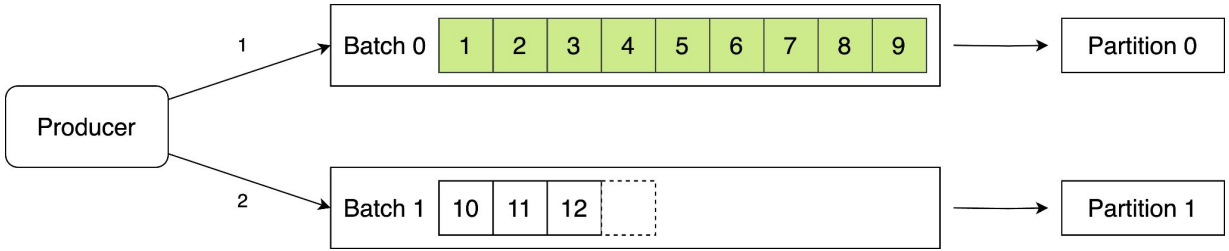
5.1. Old Default Partitioner

- In < Kafka 2.4.0, if the partition and key in records are not specific, then the records will be sent using a round-robin strategy.
- Producers don’t immediately send records, instead placing them into partition-specific batches to be sent later.
- Drawbacks:
  - More small batches will be sent
    → more requests, more load on brokers
  - **May add latency**

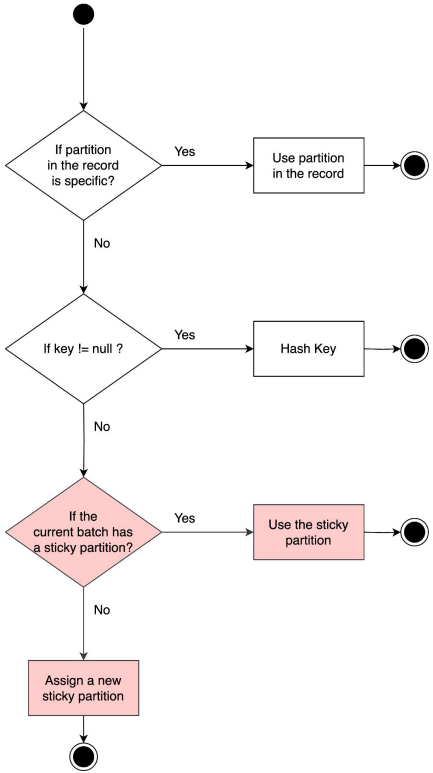


5.2. Sticky Partitioner

- Kafka 2.4.0 added the sticky partitioner.
- **Picking a single partition to send all non-keyed records.**
- Once the batch is filled, randomly chooses and “sticks” to a new partition.
- The sticky partitioner still results in an even distribution of records.
- It is like a “per-batch” round robin or “**eventually even**” approach.



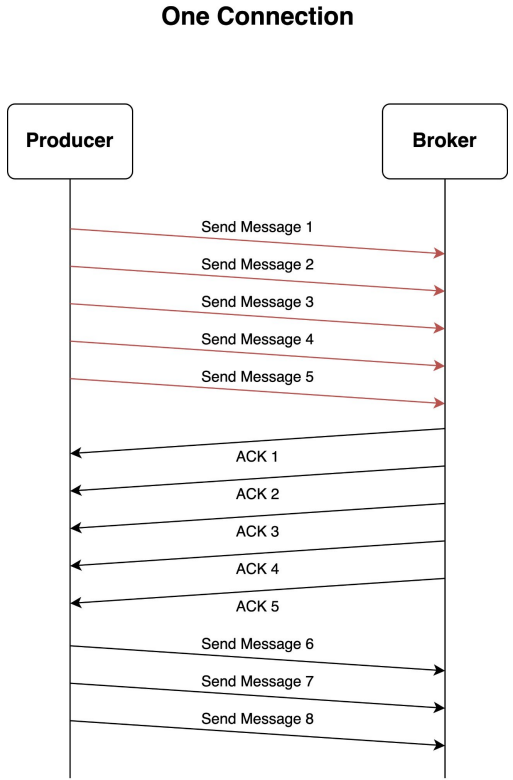
5.3. New Default Partitioner



6. Max In-Flight Requests

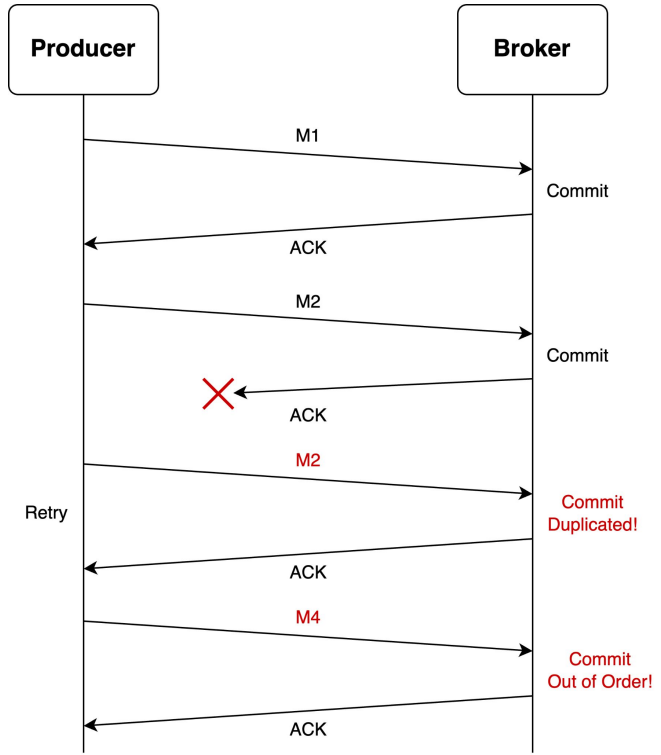
6. Max In-Flight Requests

- max.in.flight.requests.per.connection: **an upper bound on the number of unacknowledged requests** on a single connection before the producer waits for the acks.
- The default value = 5 (Kafka >= 1.0)
- Purpose: improve the throughput of producer, not improve network bandwidth.



7.0. Problems

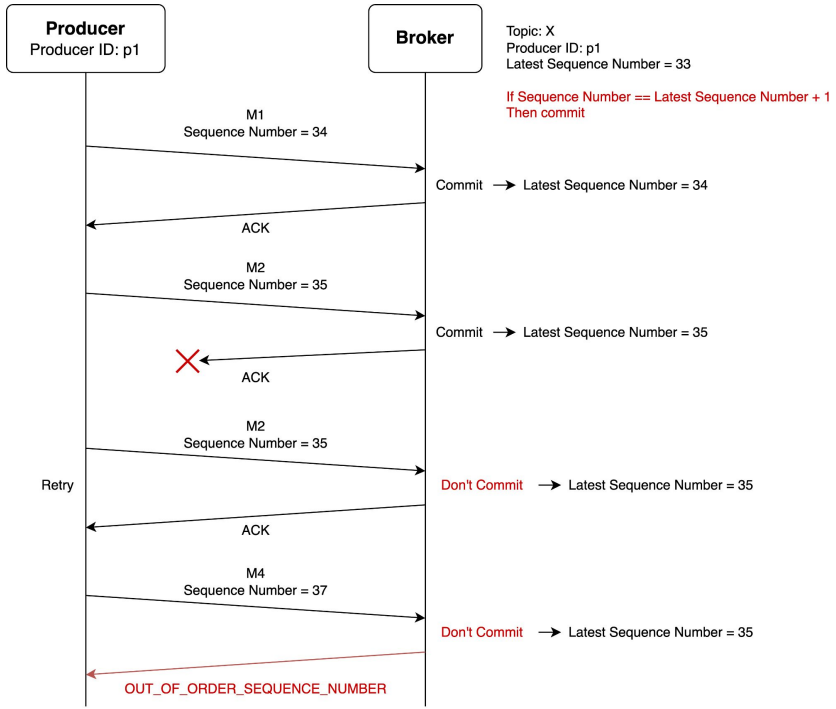
- When working with collections,
  What should problems be considered first?
  - **Record duplication**
  - **Out of order**



7. Idempotent

7.1. Idempotent

- Idempotence: the operation can be applied multiple times without changing the result.
- How a idempotent producer works?
  - A **unique producer ID** (PID) is assigned by a broker.
  - A monotonically **increasing sequence number** to each record.



7.2. Implementation

- **enable.idempotent = true** (recommended)
  - False: default before Kafka 3.0
  - True: default since Kafka 3.0
- Come with:
  - **acks=all**
  - retries=INT_MAX
  - max.in.flight.requests=5
- Note: Idempotent producers only resolve ordering issues on producer side.

